

EngageIQ

Smart Engagement Opportunity Scorer

BAX-423 Big Data | Technical Brief

Saliha Askin Sonmez

1. System Architecture

EngageIQ is an end-to-end ML pipeline using local Sentence-BERT embeddings, FAISS retrieval, and multi-signal ranking that discovers, scores, and learns from high-value engagement opportunities across GitHub and Hacker News. The pipeline has six stages:

Stage 1 - Multi-Source Ingestion and Streaming

Three collectors (`collect_github_events.py`, `collect_hackernews_posts.py`) poll the GitHub REST API v3, GH Archive, and Hacker News API. Records are streamed into a raw landing zone and deduplicated by (source, id) and URL before storage.

Stage 2 - Merge, Deduplication and Feature Engineering

`merge.py` runs a 9-stage pipeline: schema standardization to merge to deduplication (source+id key, then URL key) to domain classification to topic classification (15 domains via keyword matching) to feature engineering (engagement_score, freshness, age_days) to offline snapshot export. Final dataset: **15,653 records** across all 15 required domains.

Stage 3 - Content Embedding (Sentence-BERT)

`generate_embeddings.py` encodes each opportunity title + description + domain + source as a dense **384-dimensional vector** using `sentence-transformers/all-MiniLM-L6-v2`. User interest profiles are encoded the same way, placing users and opportunities in the same vector space.

Stage 4 - FAISS Retrieval and Multi-Stage Ranking

`retrieve_candidates.py` builds a FAISS IndexFlatL2 index and retrieves the top-100 candidates per query. `ranking.py` applies a weighted final score: similarity (0.45) + engagement_score (0.20) + freshness_score (0.15) + trend_score (0.10) + effort_score (0.10). Diversity re-ranking uses simple domain-bonus or Maximal Marginal Relevance (MMR).

Stage 5 - Adaptive Learning

`adaptive_learning.py` updates the user embedding vector online after each feedback event (engage/bookmark/skip). NDCG@10 is tracked across 50+ rounds to demonstrate measurable improvement.

Stage 6 - Batch Analytics (PySpark) and Dashboard

`analytics_spark.py` runs distributed analytics (active communities, volume over time, category distributions, trending topics via TF-IDF). The Streamlit dashboard integrates the FAISS pipeline at runtime with ranked opportunities, "Why this?" explanations, writing suggestions, feedback controls, and a downloadable weekly brief.

2. BAX-423 Technique Choices

Technique 1: Content Embeddings + Approximate Nearest Neighbor Search (FAISS)

Rationale: Raw keyword matching fails to capture semantic similarity ("LLM fine-tuning" vs "language model training"). Sentence-BERT produces dense contextual embeddings aligning opportunity text and user profiles in a shared vector space. FAISS IndexFlatL2 provides millisecond-latency exact L2 nearest-neighbor search over the full 15,653-vector index.

- **Model:** sentence-transformers/all-MiniLM-L6-v2 -- 384-dim, 80 MB, optimized for semantic similarity.
- **Index:** FAISS IndexFlatL2 -- exact L2 search, no quantization error at this dataset size.
- **Fallback:** NumPy L2 exact search when FAISS is unavailable (same correctness, slower).

Technique 2: Multi-Stage Ranking with Diversity Re-Ranking

Rationale: Retrieval alone does not account for community health, recency, or required effort. A multi-stage pipeline separates candidate generation (FAISS top-100) from engagement scoring and diversity re-ranking, enabling independent optimization.

Ranking weights: similarity 0.45 | engagement 0.20 | freshness 0.15 | trend 0.10 | effort 0.10

- **Simple re-rank:** +0.05 bonus to first candidate per domain, then re-sort.
- **MMR:** Greedy Maximal Marginal Relevance -- relevance vs. diversity trade-off (lambda=0.7).

Technique 3: Adaptive Learning from Feedback

Rationale: User interests evolve. Online update of the user embedding after each feedback event enables personalization without retraining.

Update rule: $\text{new_vector} = \text{normalize}(w_{\text{user}} \times \text{user_vec} + w_{\text{item}} \times \text{item_vec})$

- **engage:** $w_{\text{user}}=0.90, w_{\text{item}}=+0.10$
- **bookmark:** $w_{\text{user}}=0.93, w_{\text{item}}=+0.07$
- **skip:** $w_{\text{user}}=0.95, w_{\text{item}}=-0.05$

Benchmark Results -- Ranking Evaluation (NDCG@10)

Model	NDCG@10
Similarity Only (baseline)	0.7173
Similarity + Engagement	0.3278
Full Scoring Model	0.4095
Full Model + MMR Re-ranking	0.4095
Post-pipeline (reranked_score)	0.9012

Note: The drop from similarity-only reflects proxy label design -- proxy labels weight similarity at 0.5, so adding orthogonal signals lowers proxy alignment while improving real diversity. The post-pipeline NDCG@10 of 0.9012 confirms the full pipeline ranks high-quality items first.

Adaptive Learning Results (50 rounds, user-alice)

Metric	Value
Initial NDCG@10	0.8524

Final NDCG@10	1.0000
Improvement	+17.32%
Learning demonstrated	True
Feedback distribution	engage: 40 bookmark: 6 skip: 4

3. Test Persona Results

Capability	Sofia ML Student	David DevOps	Lina Journalist	Raj Founder
1. Multi-Source Ingestion & Streaming	PASS	PASS	PASS	PASS
2. Content Embedding & Similarity	PASS	PASS	PASS	PASS
3. Engagement Scoring & Ranking	PASS	PASS	PASS	PASS
4. Adaptive Learning from Feedback	PASS	PASS	PASS	PASS
5. Batch Analytics & Trend Detection	PASS	PASS	PASS	PASS
6. Dashboard & Engagement Brief	PASS	PASS	PASS	PASS

Persona 1 -- Sofia ML Student (Portfolio Builder)

Profile: Machine learning, NLP, data pipelines; Python, pandas; finds beginner-friendly GitHub repos.

Result: PASS. Top-10 includes GitHub repos in Machine Learning and AI Research domains. Zero Embedded Systems or GameDev repos appear. Writing suggestions target the "ask a thoughtful question" goal.

Persona 2 -- David DevOps Engineer (Niche Community)

Profile: Kubernetes, Terraform, CI/CD, observability; expert commentary opportunities.

Result: PASS. Top-10 is Kubernetes/infra-focused (DevOps/K8s domain). Semantic embedding separates "kubernetes helm chart" from general web dev. Recommends repos with high engagement and moderate contributor count. Items are discussion-oriented.

Persona 3 -- Lina Data Journalist (Trend Spotter)

Profile: Trending repos, viral discussions, emerging tools; breadth over depth; 10 hrs/week.

Result: PASS. trend_score (comments / max(age_days, 1)) emphasizes high-velocity recent items. Batch analytics show category distributions and volume over time. Weekly brief highlights rising opportunities.

Persona 4 -- Raj Startup Founder (Marketing-Focused)

Profile: Developer productivity, APIs, CLI tools, open-source business models; Tech blogs.

Result: PASS. Top-10 is developer tools and B2B SaaS-focused. Blog items are discussion threads. After simulated skip feedback, adaptive learning shifts the user vector away from low-engagement items (w_item=-0.05 for skip events in the online update rule).

4. Pipeline Design Notes

Data Ingestion

All four APIs were used: **GitHub REST API v3** (repository events, issues, stars), **GH Archive** (historical event data for trend computation) and **Hacker News API** (stories, scores, comment counts). Records are standardized to a common schema (id, source, title, text, url, score, num_comments, created_utc, domain) before merge.

Deduplication Strategy

Two-pass deduplication: (1) source + record_id composite key, keeping the highest-engagement duplicate; (2) normalized URL key for cross-source deduplication. Removed duplicates are logged in snapshot metadata.

Domain Coverage (15,653 records)

Domain	Count	Domain	Count
Frontend (React/Web)	2,237	AI Research	629
Trending Open-Source	1,864	Blockchain	599
Developer Tools	1,533	Mobile Dev (iOS/Flutter)	495
DevOps/K8s	1,091	Python Data Eng	414
Machine Learning	1,058	B2B SaaS	353
Cybersecurity	782	Cloud APIs	332
Unknown/Uncategorized	3,864	GameDev / Embedded / Beginner	402

Embedding Pipeline

Opportunity text template: "Title: {title} Description: {text} Domain: {domain} Source: {source}". **User profile template:** "Interests: {interests}. Expertise: {expertise}. Topics: {domains}.". Both use the same SentenceTransformer model so profiles and opportunities share the same 384-dimensional vector space.

5. System Limitations

- **Beginner Coding underrepresentation:** Only 30 records (0.2% of dataset). Fix: target r/learnprogramming, r/cs50, and GitHub "good first issue" search endpoint.
- **No live streaming:** Current collectors are batch pollers. A production system would use Kafka topics with PRAW streaming and GitHub webhooks.
- **Proxy NDCG labels:** Evaluation uses proxy relevance labels (similarity-weighted) rather than human judgments, inflating similarity-only NDCG.
- **User embedding cold start:** New users without pre-computed embeddings incur real-time SBERT encoding (~200ms latency on first query, cached after).
- **Single-user adaptive learning:** Updates one user vector at a time. A production system would persist updated vectors across sessions.